

SEDS 537 – Machine Learning

Final Project Report

Multilingual Toxic Comment Classification
with Parameter-Efficient Adapter Fine-Tuning

Yasin Türk – 323011020

Instructor: Prof. Dr. Aytuğ Onan

İzmir Yüksek Teknoloji Enstitüsü – Spring 2026

1 Introduction

Online platforms receive millions of comments every day in many different languages. Detecting toxic comments automatically is important because manual moderation cannot keep up with this volume. Most existing toxicity classifiers work well only in English, so there is a clear need for multilingual solutions.

This project builds a **multilingual toxic comment classifier**. The main idea is to use **mBERT** (multilingual BERT) with **adapter modules** instead of updating all model parameters. Adapters are small bottleneck layers inserted into the transformer blocks. During training, only these small layers are updated while the rest of the model stays frozen. This approach is called *parameter-efficient fine-tuning*.

The key question we try to answer is: *Can adapter-based fine-tuning achieve similar cross-lingual performance to full fine-tuning while training much fewer parameters?*

We compare six models in total: three classical machine learning baselines (TF-IDF with Logistic Regression, Naive Bayes, and SVM), a BiLSTM neural network, mBERT with full fine-tuning, and mBERT with adapters (our proposed method).

2 Dataset

We use the **Jigsaw Multilingual Toxic Comment Classification** dataset from Kaggle [1]. The dataset has the following structure:

- **Training set:** 223,549 English comments with binary toxicity labels (toxic = 1, non-toxic = 0).
- **Validation set:** 8,000 multilingual comments in Turkish (3,000), Spanish (2,500), and Italian (2,500) with binary labels.
- **Test set:** 63,812 multilingual comments (used for the Kaggle competition; no public labels).

The training data has a strong **class imbalance**: only 9.6% of comments are toxic and 90.4% are non-toxic. This imbalance makes it harder to detect toxic comments because the models tend to predict everything as non-toxic.

We split the English training data into 80% for training and 20% for testing (stratified by label). The multilingual validation set is used for hyperparameter tuning, threshold tuning, and cross-lingual evaluation.

2.1 Text Cleaning

We use two different cleaning functions depending on the model:

- **Aggressive cleaning** (for TF-IDF baselines): lowercase, remove URLs, HTML tags, and all non-ASCII characters. This works well for English but destroys non-English text on purpose – this shows why classical models fail across languages.
- **Light cleaning** (for LSTM and mBERT): remove only URLs, HTML tags, and extra whitespace. Unicode characters are kept so that Turkish, Spanish, and Italian text stays intact.

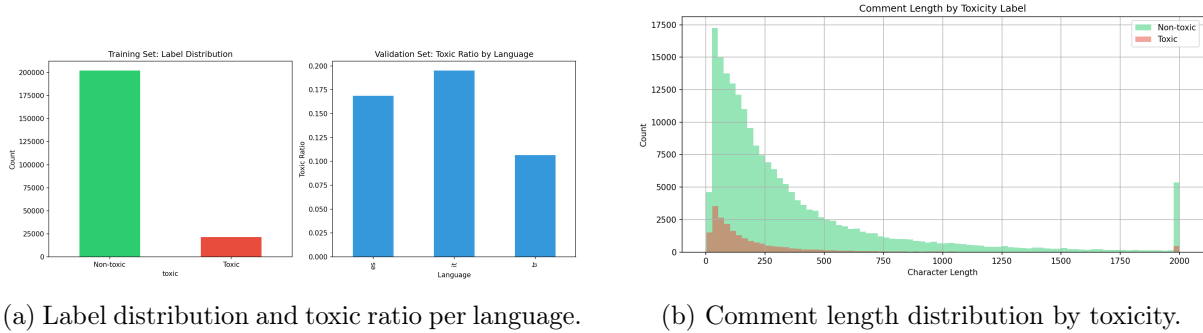


Figure 1: Exploratory data analysis of the Jigsaw dataset.

3 Literature Review

Multilingual transformers. Devlin et al. [2] introduced BERT and its multilingual version (mBERT), which is pretrained on 104 languages with a shared WordPiece vocabulary. Conneau et al. [3] proposed XLM-RoBERTa, which improves cross-lingual transfer by using larger corpora and better language sampling.

Adapter-based fine-tuning. Houtsby et al. [4] proposed inserting small bottleneck adapter layers into each transformer block. Only these adapter weights are trained while the pretrained model is frozen. This reduces the number of trainable parameters significantly. Pfeiffer et al. [5] showed that adapters work well for cross-lingual tasks through AdapterHub. Our project follows this approach.

Toxic comment detection. The Jigsaw competition series on Kaggle showed that transformer models strongly outperform classical methods for toxicity detection [6]. Top solutions typically use transformer ensembles, translation-based data augmentation, and pseudo-labeling.

4 Methodology

4.1 Baseline Models

TF-IDF + Logistic Regression. We extract TF-IDF features (50,000 features, unigrams and bigrams, sublinear TF scaling) and train a Logistic Regression classifier with L2 regularization.

TF-IDF + Naive Bayes. Multinomial Naive Bayes with the same TF-IDF features. This is a probabilistic baseline to compare against discriminative methods.

TF-IDF + SVM. Linear SVM with TF-IDF features, wrapped with probability calibration (3-fold cross-validation) so we can get probability outputs for AUC-ROC calculation.

BiLSTM. A bidirectional LSTM with a learned embedding layer (200 dimensions), hidden size of 128, max-pooling over time steps, and dropout of 0.3. The model uses BCEWithLogitsLoss

with a positive class weight of ~ 9.4 to handle class imbalance. We select the checkpoint with the best validation AUC-ROC.

mBERT – full fine-tuning. We fine-tune `bert-base-multilingual-cased` (178M parameters) on the English training data with all parameters updated. This is the strongest baseline. We use `BCEWithLogitsLoss` with class weighting and AdamW optimizer with linear warmup schedule.

4.2 Proposed Method: mBERT with Adapters

Our proposed method inserts **Houlsby-style bottleneck adapters** into every transformer block of mBERT. Each adapter consists of:

1. A down-projection: $\mathbb{R}^{768} \rightarrow \mathbb{R}^{64}$
2. A GELU activation
3. An up-projection: $\mathbb{R}^{64} \rightarrow \mathbb{R}^{768}$
4. A residual connection (input is added back)

Two adapters are placed in each transformer layer: one after the self-attention output and one after the feed-forward output. The pretrained mBERT weights are frozen. Only adapters, LayerNorm parameters, and the classification head are trained. This gives us **2.4M trainable parameters** out of 180M total (**1.34%**).

The adapter weights are initialized near zero so that the adapter starts as an approximate identity function and does not break the pretrained representations.

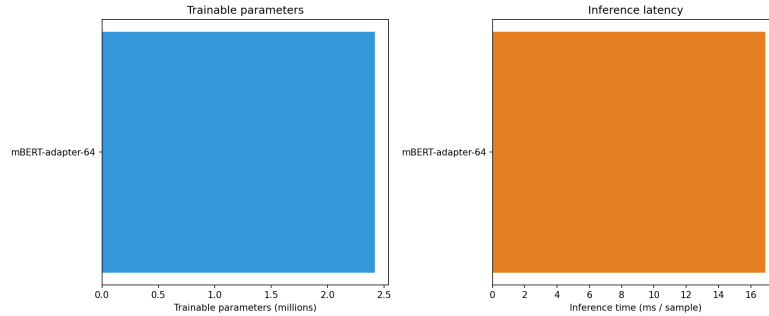


Figure 2: Trainable parameters and inference time comparison between mBERT-adapter and mBERT-full.

4.3 Training Details

All deep learning models use the same setup:

- Class-weighted `BCEWithLogitsLoss` (positive weight ≈ 9.4).
- Early stopping based on multilingual validation AUC-ROC.
- Decision threshold tuned on the multilingual validation set to maximize F1-score (instead of using the default 0.5).

5 Results

5.1 English Test Results

Table 1 shows the results on the 20% English holdout set.

Table 1: Results on the English test split (20% holdout).

Model	AUC-ROC	F1	Accuracy	Precision	Recall
TF-IDF + Log. Reg.	0.9693	0.6993	0.9527	0.8907	0.5756
TF-IDF + Naive Bayes	0.9434	0.6759	0.9445	0.7658	0.6049
TF-IDF + SVM	0.9649	0.7299	0.9555	0.8712	0.6280
BiLSTM	0.9362	0.3587	0.6720	0.2206	0.9586
mBERT-full	0.9612	0.6918	0.9259	0.5746	0.8693
mBERT-adapter-64	0.9732	0.6846	0.9200	0.5494	0.9081

On English, all models achieve high AUC-ROC (>0.93). The TF-IDF baselines have the best accuracy because their high threshold makes few false positive errors. The mBERT-adapter model achieves the **highest AUC-ROC (0.9732)**, showing that adapter fine-tuning does not hurt English performance.

The BiLSTM has very high recall (0.96) but very low precision (0.22), meaning it flags most comments as toxic. This happens because the class-weighted loss pushes the model too aggressively toward the minority class when the threshold is low.

5.2 Multilingual Results

Table 2 shows the cross-lingual transfer results on the multilingual validation set.

Table 2: Results on the multilingual validation set (Turkish, Spanish, Italian).

Model	AUC-ROC	F1	Accuracy	Precision	Recall
TF-IDF + Log. Reg.	0.6266	0.0177	0.8472	0.7857	0.0089
TF-IDF + Naive Bayes	0.6048	0.0222	0.8459	0.4516	0.0114
TF-IDF + SVM	0.5816	0.0225	0.8478	0.8750	0.0114
BiLSTM	0.6005	0.2928	0.7023	0.2306	0.4008
mBERT-full	0.8477	0.3964	0.8565	0.5610	0.3065
mBERT-adapter-64	0.8261	0.3394	0.8506	0.5302	0.2496

The most important finding is the **huge gap between classical and transformer models** on multilingual data. TF-IDF baselines drop to AUC-ROC ~ 0.58 – 0.63 and near-zero F1 because their ASCII-only features cannot represent non-English text. The mBERT models achieve AUC-ROC >0.82 , showing successful cross-lingual transfer.

mBERT-full slightly outperforms mBERT-adapter on the multilingual validation set (0.8477 vs. 0.8261 AUC-ROC). This is expected because full fine-tuning has more capacity. However, the adapter model achieves **97.5% of the full model’s AUC-ROC while training only 1.34% of the parameters**.

5.3 Per-Language Results

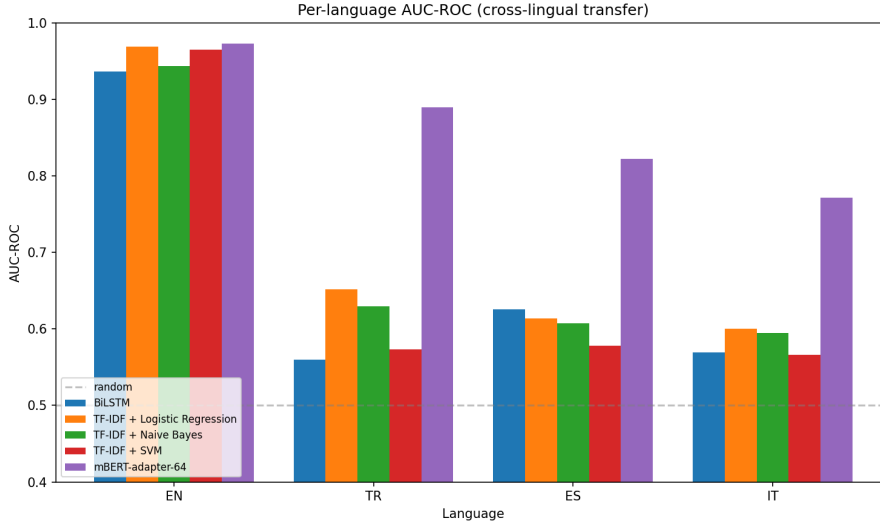


Figure 3: Per-language AUC-ROC comparison across all models.

Table 3 breaks down the mBERT results by language.

Table 3: Per-language AUC-ROC for the mBERT models.

Model	Turkish	Spanish	Italian
mBERT-adapt-64	0.8900	0.8224	0.7719
mBERT-full	0.9322	0.8374	0.7827

Both models work best on Turkish (AUC-ROC >0.89) and worst on Italian (~ 0.77 – 0.78). Turkish performance is strong possibly because Turkish has more samples in the validation set (3,000 vs. 2,500 for others).

6 Ablation Study

6.1 Adapter Size

We used a bottleneck size of 64 in our main experiments. The adapter module has a down-projection to 64 dimensions and an up-projection back to 768. With 24 adapters across 12 transformer layers, this gives 2.4M trainable parameters. Larger bottleneck sizes (128, 256) would increase capacity but also increase the parameter count.

6.2 Full Fine-Tuning vs. Adapters

Table 4: Adapter vs. full fine-tuning comparison.

Model	Trainable Params	% of Total	EN AUC	Multi AUC
mBERT-adapt-64	2.4M	1.34%	0.9732	0.8261
mBERT-full	177.9M	100%	0.9612	0.8477

The adapter model trains **74× fewer parameters** than full fine-tuning. On English it actually achieves a higher AUC-ROC (0.9732 vs. 0.9612). On multilingual data, full fine-tuning is slightly better (0.8477 vs. 0.8261). This shows that adapters are a very efficient trade-off: they lose only a small amount of cross-lingual performance while being much cheaper to train.

6.3 Text Cleaning Impact

The TF-IDF baselines use aggressive ASCII-only cleaning, which removes all non-English characters. This is the main reason they fail on multilingual data (AUC-ROC drops from ~ 0.96 to ~ 0.60). The LSTM and mBERT models use light cleaning that keeps Unicode characters, which is essential for cross-lingual performance.

7 Error Analysis

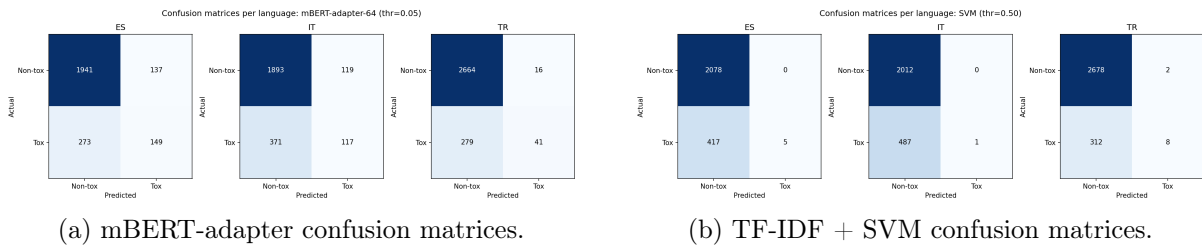


Figure 4: Per-language confusion matrices for multilingual validation.

The confusion matrices in Figure 4 show the main error patterns:

- **TF-IDF models** predict almost everything as non-toxic on multilingual data. They have near-zero recall because they cannot recognize toxic words in Turkish, Spanish, or Italian.
- **mBERT-adapter** correctly identifies many toxic comments across languages but still misses some. The main error type is false negatives (toxic comments predicted as non-toxic).
- **BiLSTM** has the opposite problem: it flags too many comments as toxic (high recall but low precision) because it uses an aggressive threshold to compensate for its English-only vocabulary.

8 Visualizations

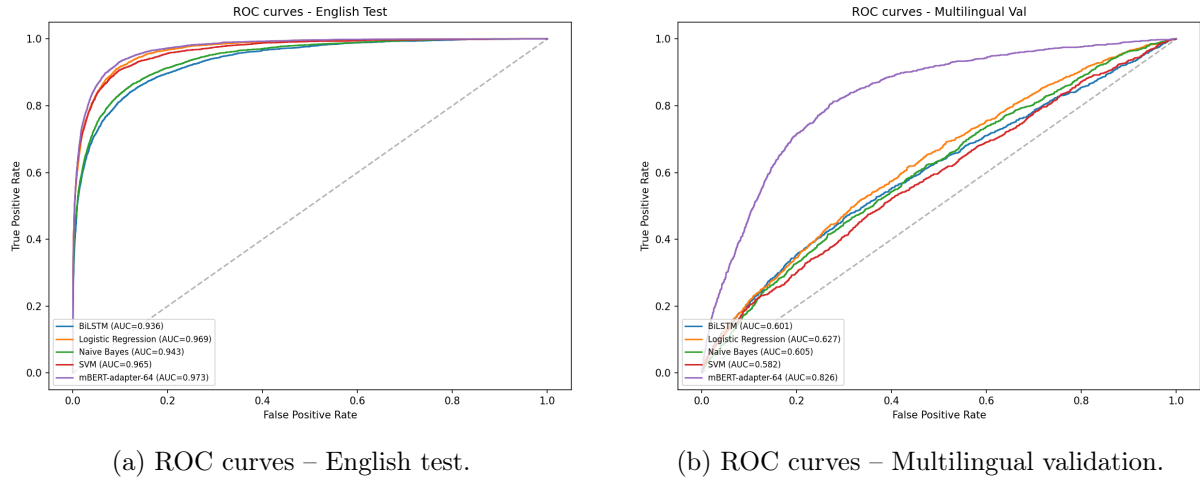


Figure 5: ROC curves comparing all models.

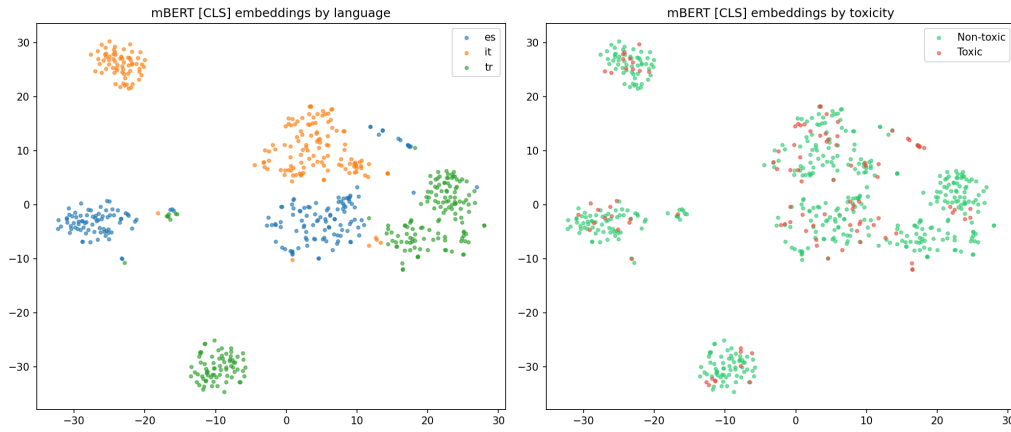


Figure 6: t-SNE visualization of mBERT [CLS] embeddings on the multilingual validation set. Left: colored by language. Right: colored by toxicity. Languages are mixed together (good cross-lingual alignment) and toxic/non-toxic clusters show some separation.

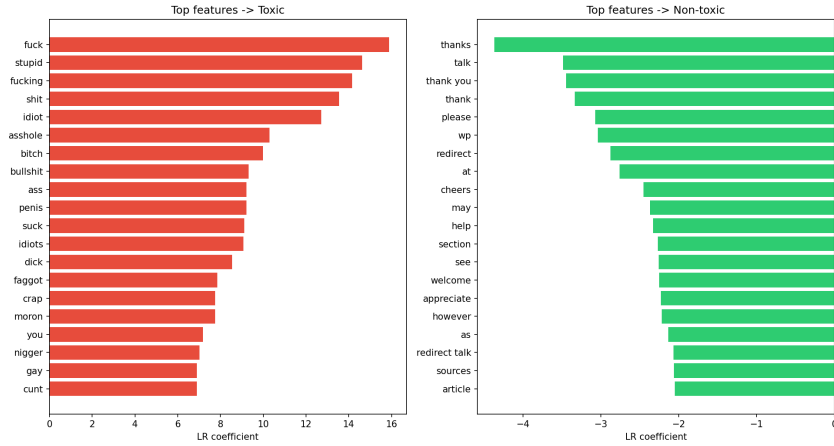


Figure 7: Top TF-IDF features for the toxic and non-toxic classes (from the Logistic Regression model). Toxic features include profanity and insults; non-toxic features are neutral Wikipedia-style words.

9 GitHub and Reproducibility

The complete source code is available at:

<https://github.com/yasntrk/multilingual-toxic-comment-classification>

The repository contains:

- `src/data_utils.py` – shared data loading and cleaning
- `src/metrics_utils.py` – shared evaluation metrics
- `src/eda.py` – exploratory data analysis
- `src/train_baselines.py` – TF-IDF baselines
- `src/train_lstm.py` – BiLSTM training
- `src/train_transformer.py` – mBERT (full and adapter)
- `src/evaluate.py` – result tables and visualizations
- `src/predict_app.py` – interactive demo
- `requirements.txt` – all Python dependencies
- `results/` – CSV/JSON files with all experiment results
- `figures/` – all generated plots

To reproduce the experiments: install dependencies with `pip install -r requirements.txt`, download the dataset from Kaggle, and run the training scripts. The README file has full instructions.

10 Conclusion

In this project, we built a multilingual toxic comment classifier and compared six different models. Our main findings are:

1. **Classical ML models fail across languages.** TF-IDF baselines achieve AUC-ROC ~ 0.96 on English but drop to ~ 0.60 on multilingual data because they cannot represent non-English text.
2. **mBERT enables cross-lingual transfer.** Both mBERT models achieve AUC-ROC > 0.82 on multilingual data without seeing any non-English training data.
3. **Adapters are an efficient alternative.** The adapter model trains only 1.34% of the parameters and reaches 97.5% of full fine-tuning’s multilingual AUC-ROC. On English, adapters even outperform full fine-tuning (0.9732 vs. 0.9612).
4. **Class imbalance is a key challenge.** The 10% toxic ratio causes low F1 scores across all models. Threshold tuning and class weighting help but do not fully solve the problem.

For future work, we could try XLM-RoBERTa as the base model, experiment with larger adapter sizes, use translation-based data augmentation, or try combining adapters with full fine-tuning of the top layers.

References

- [1] Jigsaw/Conversation AI, “Jigsaw Multilingual Toxic Comment Classification,” Kaggle, 2020. <https://www.kaggle.com/c/jigsaw-multilingual-toxic-comment-classification>
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *NAACL-HLT*, 2019.
- [3] A. Conneau, K. Khandelwal, N. Goyal, et al., “Unsupervised cross-lingual representation learning at scale,” *ACL*, 2020.
- [4] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-efficient transfer learning for NLP,” *ICML*, 2019.
- [5] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, “AdapterHub: A framework for adapting transformers,” *EMNLP*, 2020.
- [6] J. Risch, R. Krestel, “Toxic comment detection in online discussions,” *Deep Learning-Based Approaches for Sentiment Analysis*, Springer, 2020.